

Evidence Semantics and Scanner Orchestration

A public technical paper for Security Observatory by FortMilo

Luca Pacini

The front-matter fields are reference data, so a table is easier to scan than prose.

Field	Value
Publication date	11 August 2026
Evidence status	Claims are classified individually. No universal release-validation or environment-support claim is made.
External verification	Product-source claims remain self-attested while the supporting source is private.
Document licence	CC BY 4.0, subject to the third-party material and trademark boundary in Publication and licensing.
Published source licence	Implementation-supporting source is privately maintained. Any separately published source states its own scope and licence; Apache License 2.0 applies only where explicitly stated with that material.

Public evidence identifiers. EV identifiers are stable citable labels. A later class change is recorded against the same identifier, identifiers are never recycled, and the private assurance record maps each EV label to its exact backing artefacts.

Evidence basis notice. I separate source inspection, automated tests, controlled runtime observations, exact-candidate release validation and support. One class does not silently substitute for another. Private source and runtime identifiers are retained in the private assurance record rather than reproduced here.

Abstract

Security tooling becomes unreliable when it treats execution, source availability, observed value and interpretation as one thing. Security Observatory keeps those questions separate. A completed process does not prove that every source was available, an empty result does not prove that the relevant population does not exist, and a framework mapping does not create a compliance decision.

I built the scanner contract around one hard rule: **nothing found and nothing assessed are not the same result**. The product runs inside Salesforce, uses bounded asynchronous orchestration, retains sanitised evidence in the subscriber organisation and keeps source limitations visible. A user can explicitly download a sanitised CSV; that download is a separate boundary crossing and is not hidden inside an "in-org" claim.

The current product also carries distinct presentation axes. Internal machine evidence state records what the scanner can establish from retained evidence. Customer-facing rendered evidence state is governed by the canonical terminology contract. The SBS/control layer separately carries a check-outcome axis, and operator summaries may add attention labels such as Needs review or Scanner issue. Those are different dimensions, not permission for each screen to invent an evidence-state vocabulary.

The current check-outcome fallback is deliberately fail-safe but coarser than the terminology contract for some limiting machine states. It does not manufacture a favourable result, but it can collapse attempted-unavailable evidence and genuine non-assessment into the same attention outcome. Section 14 records that limitation rather than pretending the current mapping is already fully aligned.

At publication, selected false-zero and scanner-failure paths have been remediated and revalidated to the evidence classes stated in Section 3. Running-user access behaviour, complete raw-IP exclusion across every retained shape, clean-install proof and environment compatibility remain bounded separately.

1. The problem is not the count

A security dashboard can make a bad assessment look reassuring if it compresses execution, source availability and result into one number.

A displayed zero might mean the source was queried successfully and returned no matching rows. It might also mean the source could not be read. Those two situations are opposites from an evidence perspective, even though a careless implementation can make them look identical.

I therefore keep four dimensions separate. **Execution** records whether the scanner or orchestration step reached a known state. **Availability** records whether the required source was accessible and complete enough for the question. **Value** records what a successfully assessed source returned. **Interpretation** limits what the product is allowed to say about that result.

The losing design was simpler: persist a count and infer the rest later. I rejected it because once a failed acquisition has become a zero, the original uncertainty is gone. The renderer cannot recover evidence that was never retained.

2. Evidence states, check outcomes and current product presentation

The current implementation carries more than one axis. The scanner-family evidence service uses eight literal internal machine identifiers. The canonical terminology contract separately defines rendered evidence states for customer-facing evidence. The SBS/control layer also carries a check-outcome axis. Coverage, Severity and completeness remain separate again. These dimensions answer different questions and must not be collapsed.

Internal identifier	Meaning in the reviewed implementation
observed	Usable family evidence was returned; at least one retained metric is observed rather than zero, or an explicit structured status resolves to observed.
zero_observed	The family was assessed and one or more returned metrics are zero, with no higher-priority limiting state.
not_returned	The family was planned for the scan but returned no metric, structured-status or coverage output from which a stronger state could be established.
unsupported	An explicit structured family status records that the relevant path or capability is unsupported.
skipped	The structured scan plan omitted the family; the family was not attempted for that run.
scanner_error	The family execution failed and only a bounded, approved failure category is retained for operator guidance.
unavailable	The assessment was attempted or coverage evidence exists, but a required source, dependency or coverage condition prevented usable complete evidence.
unknown	The service cannot establish a more specific state from reliable retained inputs; retained-evidence read failure also fails closed to internal unknown with a bounded reason.

The internal identifiers above are implementation states only. They are not an alternative customer-facing evidence-state vocabulary. The current public model is easier to reason about when each axis is named explicitly.

Axis	Purpose	Current vocabulary / boundary
Machine evidence state	Internal scanner/evidence condition.	observed, zero_observed, not_returned, unsupported, skipped, scanner_error, unavailable, unknown. Internal only.
Rendered evidence state	What the product can truthfully state about the evidence condition.	Contextual metric or finding label; None found; Unavailable; Not assessed; Not retained at this evidence level; Not captured; Not applicable.
Check / control outcome	What the SBS/control assessment observed or failed to establish.	Risk evidence; No risk evidence surfaced; Unknown/Error; Not applicable. Aggregate unknown_error can be presented as Needs review. These are not rendered-evidence-state synonyms.
Coverage	How far or by what method the question is assessed.	Automated; Partial Evidence; Manual Required; Not Covered; Extended Check.
Severity	Priority of retained risk context.	Critical; High; Moderate.
Completeness	Whether otherwise usable evidence is complete for the intended scope.	Partial is a qualifier, not a rendered evidence state.

Scanner issue is an operator warning label used when scanner-family issue evidence needs attention. It is not a rendered evidence state and it is not the control-outcome value itself.

Where a metric surface has successfully assessed zero, a conforming metric presentation can show a numeric value together with None found. The SBS/control outcome can separately say No risk evidence surfaced. Those statements can coexist because value, rendered evidence state and check outcome answer different questions.

The current mapping between machine state and check outcome is deliberately fail-safe but semantically coarse. Several limiting machine states, including unavailable, skipped, not_returned, unsupported and internal unknown, can resolve to Unknown/Error. That fallback does not create a false clean result, but it does not preserve every distinction that the rendered-evidence terminology contract keeps separate - especially attempted-unavailable evidence versus genuine non-assessment. The separate EV-04 retained-evidence read warning may use the reserved customer-facing token Unknown; that bounded warning is not the general check-outcome fallback and is not a general rendered evidence state.

I therefore do not publish a one-to-one mapping from every internal machine value to one canonical rendered state. The current implementation cannot support that claim for every state. The paper records the distinction and the limitation instead of upgrading the product by prose.

3. The scanner contract and the evidence transition

3.1 Current contract

When source acquisition succeeds, the scanner may retain the observed value within its defined scope. When acquisition fails, the scanner must keep the limiting evidence condition explicit and must not substitute a zero. A composite scanner may preserve independently usable evidence, but it must mark completeness as Partial rather than manufacturing a complete total.

A family exception is isolated where that can be done safely, so independent later work continues. A Queueable transaction failure outside that isolation boundary is different: the scan is marked Failed, already committed evidence is retained where the failure contract allows it, and the next segment is not enqueued. The user sees a failed scan rather than a completed partial scan. Families in segments that never ran cannot receive a successful-zero result.

Known safe failure categories may be retained as bounded reason codes with fixed next actions. Raw exception text, stack traces and query text stay out of retained evidence. Internal unknown or unclassified machine conditions remain conservative and cannot resolve to a favourable zero. Internal unknown is not a general rendered evidence state. The reserved customer-facing Unknown token is limited to the bounded EV-04 retained-evidence read warning; it does not substitute for Unavailable or Not assessed. I would rather lose diagnostic precision than retain attacker-useful detail.

3.2 What changed after the false-zero review

Review found real acquisition-failure paths in the Connected App and Sites/Experience scanner families where a failed source could still reach zero-shaped evidence. Separate reviews found two more failure modes: scanner exceptions could lose useful bounded cause information, and one retained-evidence read failure could collapse family state into unexplained internal unknown outcomes.

EV-04 accounts for that broader historical retained-read problem. The older Queueable-depth suspicion was a different question. Source inspection and one Developer Edition execution showed the intended grouped plan completing in five segments, so I retired the specific six-job hypothesis as not a defect and retained that decision as EV-09.

I fixed the confirmed product paths before I strengthened the public claims. EV-01 through EV-04 record the evidence transition. The companion delivery-method paper records the same chronology: favourable classes were frozen while the defects remained unresolved, then changed only after named remediation evidence existed.

3.3 Named evidence records

The register is reference material. Most EV records back a public claim; EV-09 documents a decision not to claim a defect after the hypothesis was disproved.

Record	Claim / decision	Evidence class	Boundary that remains
EV-01 Connected App false-zero remediation	A Connected App source failure remains unavailable; a successful genuine zero remains distinct.	Source-established and test-observed.	No runtime claim for every genuine-zero path.
EV-02 Sites/Experience unavailable-source validation	Failed Site/Experience sources retain unavailable, non-zero-shaped evidence rather than false zero.	Test-observed and runtime-observed for the controlled unavailable-source execution.	Successful-zero runtime behaviour is not claimed from that execution.
EV-03 Classified scanner-failure validation	Selected scanner exception categories retain bounded causes and safe next actions; unclassified categories fail closed.	Source-established, test-observed and runtime-observed in a controlled classified-failure smoke.	Applies only to exercised or deterministically mapped categories.
EV-04 Retained-evidence read-failure validation	A retained-evidence read failure no longer collapses every family into an unexplained favourable state; readable retained evidence is preserved where the bounded layer permits.	Source-established and test-observed; healthy-path runtime-observed.	The forced degraded read path was not manufactured in live runtime, so that branch is not called runtime-observed.
EV-05 Helper-backed CSV mitigation	The reviewed JavaScript export helper neutralises enumerated spreadsheet trigger prefixes and flattens reviewed control characters.	Source-established and test-observed.	Does not cover Apex-generated or future paths unless they use the reviewed helper and tests.
EV-06 Retained comparison implementation	Comparison requires compatible retained evidence and source-organisation context in the reviewed implementation.	Source-established.	Exact-candidate and environment-specific comparison validation remain separate.
EV-07 Raw-IP exclusion boundary	Raw-IP exclusion is a design requirement; representative helper-backed export tests redact IP-looking values.	Expected.	Representative helper-backed export tests exist, but complete retained-data and export-shape validation across every path is not established.
EV-08 Running-user access boundary	User-facing evidence must respect approved sharing, CRUD and FLS behaviour.	Expected release-gate requirement.	Exact-candidate non-administrator persona validation is still required.
EV-09 Observed orchestration decision	In the observed Developer Edition execution, the reviewed grouped scanner plan completed in five Queueable segments; the historical six-job chain-depth hypothesis was rejected.	Source-established and runtime-observed for that execution.	Documents a decision not to claim a chain-depth defect; does not prove every environment or failure mode.

EV identifiers are stable citable labels. A class change remains attached to the same identifier, and an identifier is never recycled. The private assurance record maps each label to exact source, tests, deployments and runtime observations. Because that backing source is private, implementation claims remain self-attested to an external reader.

4. Evidence classes and permitted claims

Class	What it means	What I am allowed to claim
Source-established	The behaviour is visible in reviewed source.	Implemented in the reviewed source; runtime not implied.
Test-observed	An automated test exercised the behaviour under a controlled fixture.	Observed under that fixture.
Runtime-observed	The behaviour occurred in a reviewed Salesforce execution.	Observed in that execution and environment.
Release-validated	The exact candidate passed the defined release gates.	Validated for the stated candidate and prerequisites.
Supported	The project accepts the behaviour as part of the product contract for stated environments.	Supported only within the declared scope and prerequisites.
Expected	The behaviour is required by design, but higher evidence is incomplete.	May be stated only as an expectation or release-gate requirement.
No conclusion available	I do not hold reliable evidence for a conclusion.	No positive or negative implementation, validation or support claim.

The canonical rendered state Not assessed is different: it denotes a scope condition in which the question was not assessed for that run or scope. It is not the evidence class No conclusion available.

No class is upgraded without a named artefact. For this paper, material class changes must point to an EV record, a public artefact or another named evidence record. If the backing evidence is private, I say that and keep the claim self-attested.

One runtime success is still one runtime success. It does not establish edition-wide support, clean-install behaviour or every persona. That distinction is tedious until the day it prevents a false claim.

5. Salesforce-native architecture

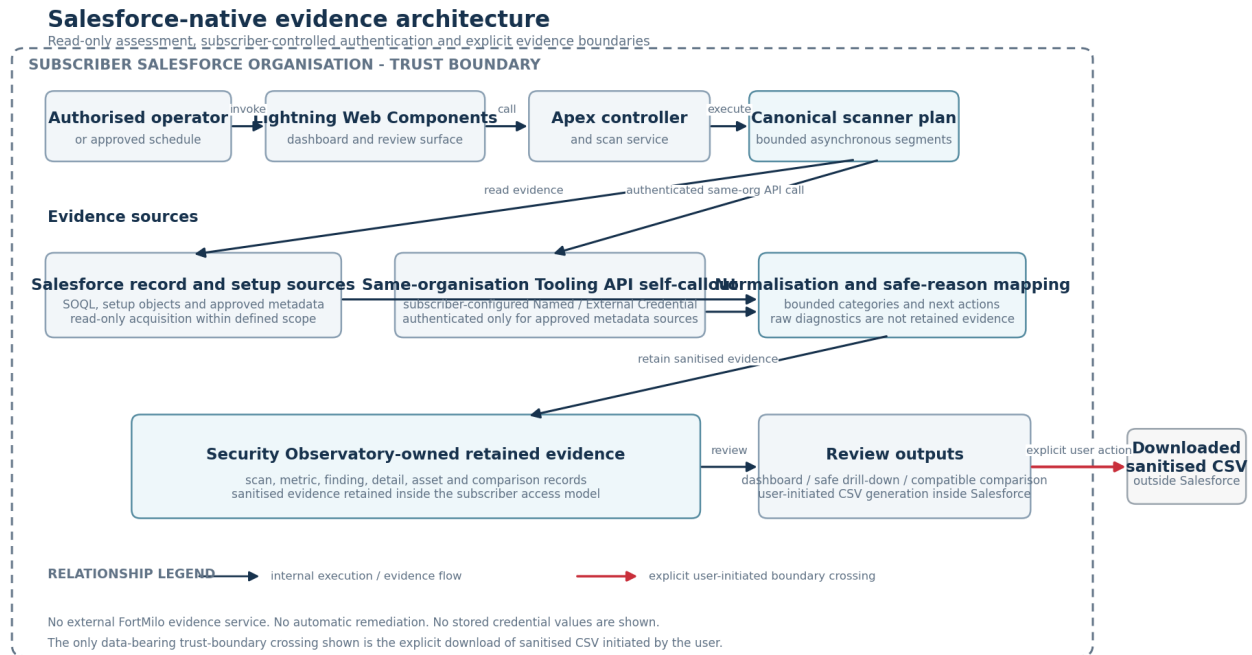


Figure 1. Salesforce-native evidence architecture. Dark arrows show internal execution or evidence flow. The red arrow is the explicit user-initiated CSV boundary crossing. The figure does not imply an external FortMilo evidence service, automatic remediation or stored credential values.

I keep the review surface, orchestration, scanners and retained product evidence in Salesforce. The Tooling API path is a same-organisation self-callout using subscriber-controlled authentication. The product does not need an external FortMilo evidence service to collect or retain scan evidence.

I also keep the scanner plan in Apex. I rejected a browser-owned scanner list because manual scans, scheduled scans and later entry points would eventually drift apart. The browser asks for a scan; Apex owns what that means.

The trust-boundary sentence needs one qualification that marketing copy often omits: user-initiated CSV download leaves Salesforce. Generation happens inside Salesforce, but once the user downloads the sanitised file, the file is outside the subscriber Salesforce trust boundary.

Running-user access is narrower than I would like to claim publicly. The design requires approved sharing, CRUD and FLS behaviour on user-facing evidence, but I will not call that environment-wide validation until the exact candidate has passed representative non-administrator persona testing.

6. Why I split the scanner plan across bounded async segments

Salesforce transaction limits make one giant assessment transaction a poor fit. The scan combines ordinary data queries, setup evidence, selected Tooling API calls and evidence persistence. I split those families across bounded asynchronous segments so a callout boundary or transaction limit in one segment does not automatically consume the whole scan.

I considered one Queueable per scanner family. It looked clean on paper and lost on transaction overhead and the risk of consuming too many chain links. I also considered one large transaction. It lost because unrelated evidence sources would share the same governor and failure boundary.

EV-09 rejects a narrower historical hypothesis: the grouped plan did not exceed the Queueable chain boundary in the observed Developer Edition run. That does not make chain depth irrelevant to a per-family fan-out design, and it does not turn the observed five-segment count into a universal product contract.

The compromise is deliberate: group compatible work, isolate callout-sensitive work, start later segments in fresh asynchronous contexts and pass forward only the orchestration context they need. Safe refactoring may change the number of segments without changing the evidence semantics.

7. Failure handling: keep the failure attached to the evidence

A scanner family may fail while later families still have independent evidence to collect. In that case I keep the family in its limiting condition and continue the independent work. The failed family does not become a successful zero just because its data structure is empty.

The Connected App false-zero defect is the concrete example. The failed acquisition used to leave an empty collection close enough to the count writer that zero-shaped evidence could escape. EV-01 now backs the narrower claim that source failure and genuine zero are distinct in the reviewed path.

Sites and Experience Cloud had a similar problem across multiple sources. EV-02 is stronger for the unavailable branch because a controlled runtime execution observed the corrected unavailable result. That runtime did not prove a successful genuine-zero branch, so the public record says so.

Scanner exceptions need a different boundary. EV-03 supports bounded categories for selected known failures and a generic fail-closed category for everything else. I rejected retaining raw exception messages because they are operationally convenient and security-poor. The admin gets a safe cause and next action; the raw diagnostic stays in the engineering channel.

Retained-evidence read failures are narrower again. EV-04 proves the layered read behaviour in source and tests, and the healthy read path has runtime observation. I did not manufacture a live degraded read failure merely to make the evidence table look stronger. The forced degraded branch remains test-observed.

8. Source availability, rendered evidence state and check outcome

The current product has distinct axes, not one status translated differently by each screen. Internal machine evidence state records what the scanner can establish from retained evidence. Rendered evidence state communicates the evidence condition under the canonical terminology contract. Check outcome communicates what the SBS/control assessment observed or failed to establish. Coverage, Severity and completeness remain separate.

A successfully assessed zero shows why the distinction matters. A metric may retain the numeric value 0 and a conforming rendered evidence-state surface may show None found. The SBS/control outcome may separately say No risk evidence surfaced. Those are not two names for one state: one is the evidence-state presentation of a successful zero, the other is the control-assessment outcome.

The current check-outcome mapping is intentionally conservative. Several unresolved or limiting machine states can collapse to Unknown/Error, while aggregate SBS presentation can label the `unknown_error` bucket Needs review. Scanner issue is a separate attention label for scanner-family issue evidence. None of those labels is a substitute for the canonical rendered evidence-state vocabulary.

This is where the current product still has a bounded semantic gap. An attempted source/read failure and a family omitted from the scan plan can both reach the same fail-safe check outcome even though the rendered-evidence model distinguishes Unavailable from Not assessed. The collapse is fail-safe rather than fail-open - it does not manufacture a clean result - but it loses an important distinction and remains a product limitation.

The design rule is therefore not 'each surface decides the words'. Different axes can carry different labels because they answer different questions. What must not happen is the collapse of those axes or the strengthening of the underlying evidence.

9. Sanitisation and export boundaries

Security Observatory retains review evidence, not a forensic dump. Tokens, session identifiers, raw secrets, private keys and certificate bodies are outside the retained review contract. Raw exception text and raw queries are also excluded from customer-facing evidence.

Raw IP handling is a narrower claim. I want the product boundary to exclude raw IP values, but I do not have complete retained-data-shape proof across every current and future path. EV-07 therefore stays at Expected for the universal exclusion claim. Representative helper-backed export tests do redact IP-looking values; that does not justify rewriting the broader class.

9.1 Spreadsheet formula-trigger mitigation

EV-05 covers the reviewed JavaScript export helper. The helper quotes cells, doubles embedded quotes, flattens tabs and line breaks, and prefixes an apostrophe when the first meaningful character is `=`, `+`, `-` or `@`.

That is useful mitigation. It is not a universal CSV-safety claim. I rejected the phrase formula-injection-safe by default because it says more than the evidence does. Apex-generated exports and future paths need their own review unless they are routed through the same helper and regression tests.

10. Subscriber-controlled Tooling API self-callout

Some metadata evidence requires the subscriber organisation to call its own Salesforce API. The approved pattern uses Named Credential and External Credential components with a subscriber-controlled principal. The package does not carry an authenticated principal on the customer's behalf.

If that prerequisite is missing or the principal lacks the required access, the evidence condition is Unavailable and current surfaces present a bounded cause and safe next action. I rejected Not assessed for this condition because the scanner did attempt an assessment and the limiting dimension is source availability, not selection.

I also reject credential probing as a verification shortcut. The Observatory is read-only and advisory. It inventories and evaluates bounded configuration evidence; it does not try to prove a credential works by using it against unrelated resources.

11. Historical evidence and comparison

A historical scan should mean what it meant when it ran. Current live state must not be used to reinterpret an older retained result.

Comparison therefore operates on retained evidence from compatible scans. The implementation represented by EV-06 checks source-organisation context and retained evidence depth before it compares results. If the evidence depth is incompatible, comparison is unavailable rather than pretending that omitted detail was resolved.

I keep this claim at Source-established. The comparison implementation exists, but this paper does not present an exact-candidate, cross-environment release-validation claim for the whole comparison surface.

12. Framework mapping supports review, not compliance

Security Observatory can attach Security Benchmark for Salesforce (SBS) references to retained evidence. I use that layer to help a reviewer organise evidence, not to manufacture a compliance result.

A control reference does not prove that the control is satisfied. Rendered evidence state, check outcome, Coverage, Severity and completeness answer different questions. For example, a family can have an Unavailable evidence condition, a fail-safe Unknown/Error check outcome, Partial Evidence coverage or a Partial completeness qualifier without any of those dimensions becoming a pass/fail compliance decision.

The retained mapping carries the registry version and local mapping revision used for the scan so later catalogue changes do not rewrite historical evidence.

Evidence class: Source-established. Reviewed local mapping metadata identifies the catalogue as mapping set M1, revision 1; a reviewed retained scan record reports SBS registry value v0.4.0. These are local version-bound mapping inputs, not certification statements. This paper does not establish the upstream asset provenance or equivalence to the current upstream release tag, and it does not publish a control-count claim.

13. Public wording discipline

These are the wording choices I use when a shorter phrase would outrun the evidence.

Avoid: "The org is secure." Prefer: "The assessed source returned the stated evidence within the documented scope."

Avoid: "No exposure exists." On rendered evidence-state surfaces, prefer None found for a successful assessed zero. On the control-outcome axis, No risk evidence surfaced remains the current bounded outcome label. Neither wording is a pass or proof that exposure cannot exist outside the assessed sources.

Avoid: treating Unknown/Error as a canonical rendered evidence state. It is a current fail-safe check-outcome fallback and does not distinguish every limiting reason. Where the rendered evidence-state axis is present, an attempted required source/read failure remains Unavailable.

Avoid: treating all missing-looking evidence as Not assessed. Distinguish unavailable source, genuine non-assessment, intentionally unretained detail, uncaptured evidence and non-applicability by the actual limiting reason.

Avoid: "Formula-injection-safe CSV export." Prefer: the reviewed helper-backed paths mitigate the enumerated formula-trigger prefixes and control characters.

Avoid: "Raw IP values are universally excluded." Prefer: raw-IP exclusion is a design requirement; complete retained-data-shape validation is still bounded by EV-07.

Avoid: "FLS-gated controls are fully validated." Prefer: running-user sharing, CRUD and FLS remain subject to EV-08 exact-candidate persona validation.

Avoid: "Evidence never leaves Salesforce." Prefer: there is no automatic external FortMilo evidence egress; a user-initiated sanitised CSV download is an explicit boundary crossing.

14. Current limitations

I have fixed the defects behind EV-01 through EV-04. That does not make the whole product release-validated. The current limits that still matter are these:

- Exact-candidate non-administrator persona validation is still required before I make an environment-wide sharing, CRUD or FLS claim.
- Complete retained-data and export-shape validation for the universal raw-IP exclusion boundary is not established.
- The forced degraded retained-evidence read branch is test-observed rather than runtime-observed.

- The current check-outcome fallback is intentionally conservative but semantically coarse: several limiting machine states can collapse to Unknown/Error. This does not produce a false clean result, but it does not preserve the distinction between every attempted-unavailable condition and genuine non-assessment on that axis.
- Like-for-like comparison is source-established here; exact-candidate and representative-environment validation remain separate.
- Clean-install and package-install proof for the exact public candidate remain separate and are not claimed here.
- A successful execution in one Salesforce organisation does not establish edition-wide compatibility.
- A Queueable transaction failure outside per-scanner isolation marks the scan Failed, retains already committed evidence where the failure contract allows it and does not enqueue later segments. Families that never ran cannot receive a successful-zero result.
- The product remains advisory and read-only; it does not remediate findings or prove that a retained OAuth authorisation is a currently usable token.
- Some Tooling-backed evidence depends on subscriber-controlled authentication and permissions.
- Product-source implementation claims remain self-attested while the supporting source is private.

These are compromises, not decorative caveats. I would like the evidence classes to be higher. They are not higher until the corresponding gate has actually been run.

15. Conclusion

A security dashboard is trustworthy only when it can explain the path from source to statement. The useful property is not the number of counts on screen. It is the refusal to turn missing evidence into favourable evidence.

That rule shaped the product and it also exposed defects in the product. The Connected App and Sites/Experience failures were not theoretical; they showed that an empty data structure can become a false zero if the scanner contract is not carried all the way to persistence. I fixed those paths, recorded the evidence transition and kept the remaining claims at their actual classes.

The same discipline applies to presentation. Different axes can legitimately carry different labels because they answer different questions. That is not permission for each screen to maintain its own evidence-state vocabulary. Machine evidence state, rendered evidence state, check outcome, Coverage, Severity and completeness must remain distinct.

A successful assessed zero is the simplest example: a metric can retain the numeric value 0, a rendered evidence-state surface can show None found, and the SBS/control outcome can say No risk evidence surfaced. Those are compatible statements because they describe different dimensions. None of them is a pass.

The current Unknown/Error fallback on the check-outcome axis remains deliberately conservative but too coarse for some limiting machine states. Recording that limitation is more useful than describing a cleaner mapping that the product does not yet ship.

Closing principle: nothing found and nothing assessed are not the same result.

References

1. Salesforce Developers, Secure Apex Classes. <https://developer.salesforce.com/docs/platform/lwc/guide/apex-security.html>
2. Salesforce Developers, Call APIs from Apex. <https://developer.salesforce.com/docs/platform/lwc/guide/data-api-calls-apex.html>
3. Salesforce Developers, Get Started with Named Credentials. <https://developer.salesforce.com/docs/platform/named-credentials/guide/get-started.html>
4. Salesforce Developers, Named Credentials Glossary. <https://developer.salesforce.com/docs/platform/named-credentials/references/named-credentials-reference/nc-glossary.html>
5. Salesforce Developers, Package Named Credentials. <https://developer.salesforce.com/docs/platform/named-credentials/guide/nc-package-credentials.html>
6. Salesforce Developers, Populate External Credential Principals. <https://developer.salesforce.com/docs/platform/named-credentials/guide/nc-populate-external-credentials.html>
7. Security Benchmark for Salesforce. <https://www.securitybenchmark.org/>
8. Security Benchmark for Salesforce documentation. <https://docs.securitybenchmark.org/>
9. Creative Commons, Attribution-ShareAlike 4.0 International. <https://creativecommons.org/licenses/by-sa/4.0/>
10. Creative Commons, Attribution 4.0 International. <https://creativecommons.org/licenses/by/4.0/>
11. Apache Software Foundation, Apache License, Version 2.0. <https://www.apache.org/licenses/LICENSE-2.0>
12. FortMilo, Orchestrating AI for Secure Software Delivery, 2026. <https://fortmilo.co.uk/documents/orchestrating-ai-for-secure-software-delivery.pdf>

Each URL is printed in full so the references remain usable in printed copies.

Publication and licensing

I am the author of this paper and trade as FortMilo. Public location: United Kingdom.

Source supporting the implementation claims in this paper is privately maintained; those claims remain self-attested unless a specific public artefact is cited. The installable package and any source code published separately are distinct publication artefacts and are not equivalent in extent. Any separately published source states its own scope and licence. Publication of source code does not by itself represent package availability, release validation or installation readiness. I retain the exact private baseline and backing artefacts for the public evidence records in the private assurance record.

Copyright 2026 Luca Pacini. This paper is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). The licence applies to FortMilo-authored text and figures in this paper. It does not relicence third-party trademarks or cited material.

Security Benchmark for Salesforce material remains subject to its own attribution and licence terms. No SBS control text, audit procedure or remediation content is reproduced or adapted in this paper.

FortMilo is a Salesforce Partner. Partner status is an organisational relationship only; it does not imply a particular partner-programme tier or entitlement, nor does it imply AppExchange listing, Salesforce security review or approval for customer distribution of Security Observatory. Security Observatory is independently developed and is not endorsed by Salesforce, Inc. Salesforce is a trademark of Salesforce, Inc.